

Guías de aprendizaje Estructuras de datos

Pilas, colas, árboles, React y Git

Ruta práctica: leer, dibujar, escribir pseudocódigo, programar y comprobar.

Guía 00. Cómo pasar de leer a escribir código

Primera lectura: entender el contrato

Antes de escribir, subraya los sustantivos del enunciado: libro, persona, nodo, menú. Se convierten en datos. Subraya los verbos: agregar, retirar, buscar, imprimir. Se convierten en métodos o interacciones de la pantalla.

Para cada método escribe cuatro cosas en una hoja:

- Entrada: qué recibe.
- Estado antes: qué elementos y punteros existen.
- Estado después: qué cambió.
- Salida: qué devuelve o muestra.

Ejemplo para retirar un libro: no recibe datos, la pila contiene A-B-C, después contiene A-B y devuelve C. Al estar vacía devuelve null y no cambia nada.

Segunda lectura: dibujar y simular

Traza tres elementos con papel. Aplica operaciones una por una. Anota tamaño, primero y último después de cada paso. Repite con cero y un elemento. Esos casos muestran casi todos los errores de referencia y de límites.

No memorices el cuerpo del método. Memoriza la regla que debe seguir siendo verdadera. Para una cola: quien entró primero y aún espera siempre está al frente. Para un árbol de búsqueda: todos los menores de un nodo están a su izquierda y todos los mayores a su derecha.

Tercera lectura: pseudocódigo propio

Escribe frases que indiquen decisiones y repeticiones. Ejemplo:

```
si la pila está vacía, devolver null
guardar el último elemento
retirarlo
devolver el elemento guardado
```

Luego traduce cada frase a JavaScript. Si una línea no se puede explicar con una frase, detente y vuelve al dibujo.

Cuarta lectura: comparación con la solución

Ejecuta las pruebas antes de mirar la solución. Si fallan, lee solo el nombre del caso que falló, dibuja ese caso y depura. Después compara tu implementación con la del repositorio. Anota diferencias de comportamiento, no solo de estilo.

Preguntas para cerrar cada reto:

1. ¿Qué pasa si la estructura está vacía?
2. ¿Qué pasa si solo tiene un elemento?

3. ¿Qué devuelve cada método?
4. ¿Qué operación cambia el orden?
5. ¿Cuál es el coste aproximado cuando hay muchos elementos?
6. ¿Puedes explicarlo sin mirar el archivo?

Plan de práctica de cuatro sesiones

- Sesión 1: pila y cola; dibujar 5 operaciones y escribir métodos básicos.
- Sesión 2: árbol binario; insertar 9 números y calcular recorridos a mano.
- Sesión 3: árbol N-ario; dibujar menú de tres niveles y escribir DFS/BFS.
- Sesión 4: React; conectar un formulario a cada estructura y verificar UI.

Usa un archivo de intento separado. Mantén la solución original como referencia y no la copies antes de completar un intento propio.

Guía 01. Pilas y colas: leer, dibujar, programar

Fuente: Clase 06 - Pilas y Colas (1).pptx, diapositivas 7 y 13.

1. Lee la idea

Una pila funciona como platos: el último que pones es el primero que retiras. Esto se llama LIFO.

Una cola funciona como una fila de personas: quien llega primero sale primero. Esto se llama FIFO.

Dibuja la pila como una columna y la cola como una fila. Escribe A, B y C en ese orden. Pregúntate qué sale con pop y qué sale con dequeue. La respuesta correcta es C para la pila y A para la cola.

2. Transforma el enunciado en datos

Reto 04: cada libro necesita nombre, ISBN, autor y editorial. Los datos de ejemplo están en ejercicios/04-pila-libros/libros.js.

Reto 05: cada persona necesita nombre, monto de retiro y fecha de llegada asignada por el sistema. Los ejemplos están en ejercicios/05-cola-cajero/personas.js. En la pantalla la fecha no se pide al usuario; la asigna el programa al entrar en la cola.

Antes de seguir, crea en papel un libro y una persona con todos sus campos.

3. Escribe los contratos de los métodos

Pila:

- push(libro): valida y pone el libro arriba; devuelve el libro agregado.
- pop(): retira y devuelve el superior; null si está vacía.
- peek(): mira el superior sin cambiar la pila.

- `size()`, `isEmpty()` y `print()`: observan el estado.

Cola:

- `enqueue(persona)`: valida y agrega al final con fecha de llegada.
- `dequeue()`: retira y devuelve quien está al frente; null si está vacía.
- `peek()`: mira el frente sin retirar.
- `size()`, `isEmpty()` y `print()`: observan el estado.

Escribe primero pruebas mentales: `push(A)`, `push(B)`, `pop()` debe devolver B; `enqueue(A)`, `enqueue(B)`, `dequeue()` debe devolver A.

4. Pseudocódigo antes de JavaScript

Pila:

`push`: agregar al final del arreglo
`pop`: retirar del final del arreglo
`peek`: leer el último sin retirarlo

Cola:

`enqueue`: agregar al final del arreglo
`dequeue`: leer el elemento en la posición inicio; aumentar inicio
`peek`: leer el elemento en la posición inicio

¿Por qué la cola usa un índice de inicio? Quitar el primer elemento de un arreglo con `shift` mueve los demás. Un índice evita ese trabajo en cada retiro. La implementación compacta el arreglo ocasionalmente para no conservar un prefijo consumido sin límite.

5. Ahora escribe tu versión

1. Crea la clase y un arreglo vacío en el constructor.
2. Implementa `size` e `isEmpty`.
3. Implementa `push/peek/pop` o `enqueue/peek/dequeue`.
4. Prueba estructura vacía, uno y tres elementos.
5. Agrega validación de campos. Un monto debe ser positivo.
6. Agrega datos de ejemplo y un demo con `console.log`.

No abras `PilaLibros.js` ni `ColaCajero.js` hasta tener un primer intento. Luego compara sus decisiones para vacíos, orden y validación.

6. Del código a la pantalla

En React, el formulario obtiene los campos. Al enviarlo llama `push` o `enqueue`. Luego se toma una nueva vista del arreglo para que React vuelva a dibujar. Un botón `pop` o `dequeue` cambia la estructura y actualiza la vista. El formulario del cajero solo pide nombre y monto; la fecha la asigna el sistema.

7. Comprueba lo aprendido

- Con A, B, C: ¿qué devuelve cada operación de retirada?
- ¿Por qué peek no cambia size?
- ¿Qué ocurre al retirar de una estructura vacía?
- Si agregas dos personas en segundos distintos, ¿cuál debe verse primero?
- Cambia el demo para retirar todos los elementos y vuelve a llenarlo.

Ejecuta `npm run demo:pila`, `npm run demo:cola` y `npm test` desde la raíz.

Guía 02. Árboles: de un dibujo a una implementación

Fuente: Clase 09 - Árboles.pptx, diapositivas 18 y 24.

1. Vocabulario que debes poder dibujar

Un árbol empieza en la raíz. Cada nodo puede tener hijos; un nodo sin hijos es una hoja. En el árbol binario hay como máximo dos hijos: izquierdo y derecho. En el árbol N-ario hay una lista de hijos y su longitud puede variar.

Dibuja el árbol binario que resulta de insertar, en este orden: 8, 3, 10, 1, 6, 14, 4, 7, 13. No lo dibujes ordenado de antemano; cada nuevo valor baja desde la raíz siguiendo comparaciones.

2. Árbol binario de búsqueda: regla central

Si el nuevo número es menor que el nodo actual, camina a la izquierda. Si es mayor, camina a la derecha. Si es igual, no insertes otro nodo. Para buscar, sigue la misma regla. No necesitas examinar todas las ramas.

Pseudocódigo de inserción:

```
si no hay raíz: el nuevo nodo será la raíz
empezar en la raíz
mientras exista un nodo actual:
    si el valor es igual: detenerse, ya existe
    elegir izquierda o derecha según la comparación
    si ese hijo está vacío: conectar ahí el nuevo nodo y terminar
    si no: continuar desde ese hijo
```

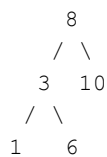
Pseudocódigo de búsqueda:

```
empezar en la raíz
mientras exista un nodo actual:
    si su valor es el buscado: devolver verdadero
    avanzar a izquierda o derecha según la comparación
devolver falso
```

Prueba primero con árbol vacío, un valor, un duplicado y uno inexistente.

3. Recorridos: calcula antes de ejecutar

Usa este árbol pequeño:



- Preorden N-L-R: visita nodo, izquierda, derecha. Resultado: 8,3,1,6,10.
- Inorden L-N-R: izquierda, nodo, derecha. Resultado: 1,3,6,8,10.
- Postorden L-R-N: izquierda, derecha, nodo. Resultado: 1,6,3,10,8.

La recursión repite la misma regla en cada subárbol. El caso base es un nodo nulo: no añade nada al resultado. Escribe primero las tres funciones en pseudocódigo y marca con lápiz el instante en que se agrega cada valor.

El reto 08 pide imprimir los tres recorridos en consola y comprobar si existe un valor. Ejecuta `npm run demo:arbol` para verlo. En React se usa `react-d3-tree`, que necesita nodos con nombre e hijos. El método `paraD3` adapta el árbol a ese formato. Su uso se basa en la documentación oficial: <https://github.com/bkrem/react-d3-tree>

4. Árbol N-ario: de datos a menú

Cada menú contiene título, enlace, componente e hijos. El padre puede tener cualquier cantidad de submenús. En `menus.js` encontrarás un ejemplo de tres niveles. Antes de abrirlo, dibuja uno con Inicio, Estudiar, Practicar y submenús para Pilas, Colas y Recorridos.

Pseudocódigo de DFS:

```
visitar nodo actual
para cada hijo, aplicar DFS al hijo
```

Pseudocódigo de BFS:

```
poner la raíz en una lista de pendientes
tomar el primer pendiente, visitarlo
agregar sus hijos al final
repetir hasta agotar pendientes
```

DFS entra en una rama antes de pasar a otra. BFS visita primero todos los nodos de un nivel. Ejemplo: para raíz A con hijos B y C, y B con hijo D, DFS entrega A-B-D-C; BFS entrega A-B-C-D.

El reto 09 pide un sidebar en React. La función `Rama` se llama a sí misma para dibujar los hijos dentro de una lista anidada. Al elegir un enlace, el árbol busca su nodo y la pantalla muestra el contenido del componente asociado. Ejecuta `npm run demo:menus` y después abre la página Menú N-ario.

5. Tu práctica sin mirar la solución

1. Implementa `NodoBinario` con valor, izquierda y derecha.
2. Implementa `insertar` y `contiene`; prueba con el dibujo de nueve números.
3. Implementa los tres recorridos y compara con tu cálculo manual.

4. Implementa NodoMenu con un arreglo hijos y agregarHijo.
5. Implementa DFS y BFS sobre tu propio menú.
6. Por último, traduce el árbol de menús a una barra lateral React.

Preguntas de control: ¿qué pasa con un árbol vacío? ¿Por qué inorden de un árbol de búsqueda queda ordenado? ¿Qué cambia si un nodo N-ario tiene cero, uno o cinco hijos?

Guía 03. Llevar una estructura a React

1. Separa responsabilidades

La estructura de datos decide el orden y las operaciones. React decide qué mostrar y cuándo actualizar la pantalla. No implementes pop, dequeue o DFS dentro del JSX. Las clases están en las carpetas de cada ejercicio; las páginas React las importan y usan.

Una página necesita tres piezas:

- Un objeto de estructura que conserva su estado entre renders.
- Una vista en estado React para pintar el arreglo o árbol actual.
- Acciones de usuario que modifican la estructura y actualizan la vista.

2. Flujo de un formulario

El formulario pide los datos que el usuario conoce. Valida campos obligatorios. Al enviarlo:

```
prevenir el envío normal del navegador
leer los campos
llamar al método de la estructura
obtener una vista nueva
actualizar el estado React
limpiar el formulario
```

En el cajero no pidas la fecha: la asigna el programa. En el árbol binario convierte el texto a número antes de insertarlo. En una pila el botón de retirar debe desactivarse cuando está vacía.

3. Flujo de los botones de consulta

Peek, búsqueda y recorridos no modifican la estructura. Una búsqueda puede mostrar un mensaje de resultado. Los recorridos se pueden mostrar como listas de números y también en la consola del navegador.

En el árbol N-ario, el sidebar se construye con una función recursiva: dibujar nodo, y si tiene hijos, dibujar una lista que repite la misma función para cada hijo. Cada botón usa el enlace del nodo para seleccionar su contenido.

4. Práctica incremental

No empieces por toda la pantalla. En este orden:

1. Muestra solo un dato fijo de ejemplo.
2. Agrega el estado de la estructura y muestra su tamaño.

3. Agrega el formulario y verifica que cambie el tamaño.
4. Muestra todos los elementos.
5. Agrega el botón de retirada o búsqueda.
6. Comprueba casos vacíos y mensajes de error.
7. Aplica estilos al final.

Abre ejercicios/04-app-react/src/paginas y compara las cuatro páginas. Modifica un dato de ejemplo, recarga y observa qué parte de la pantalla cambia. Luego crea una quinta página de práctica sin copiar los componentes existentes.

Guía 04. Un ejercicio por rama, todos en main

El repositorio previo tiene una rama por ejercicio de listas y una rama de React. Para los nuevos retos usa la misma idea: 04-pila-libros, 05-cola-cajero, 08-arbol-binario y 09-arbol-nario. Main integra los cuatro.

Lee el estado antes de modificar

```
git status
git branch
git log --oneline -5
```

Comprueba que no haya cambios tuyos pendientes. Si los hay, decide si pertenecen al ejercicio actual antes de cambiar de rama.

Ciclo de trabajo de una rama

```
git switch main
git switch -c 04-pila-libros
# escribir y probar el ejercicio 04
git add ejercicios/04-pila-libros
git commit -m "implementa pila de libros"
git switch main
git merge 04-pila-libros
```

Repite con cada reto y su nombre de rama. Cambia los archivos y el mensaje según corresponda. Ejecuta npm test después de cada merge. Cuando haya conflictos, lee ambos lados, conserva las funciones necesarias y vuelve a probar antes de completar el merge.

Qué es commit, merge y push

Un commit guarda una versión local. Merge incorpora otra rama a la actual. Push envía commits al remoto. Por eso primero se comprueba que el ejercicio funciona localmente; después se suben la rama y main si corresponde.

```
git push -u origin 04-pila-libros
git push origin main
```

El PDF [material-clase/git-cheat-sheet-education.pdf](#) sirve como referencia rápida de comandos. No reemplaza la práctica: ejecuta los comandos en un repositorio de ensayo y explica qué cambió tras cada uno.

Ejercicio de lectura a código

Dibuja cuatro líneas, una por rama. Marca el commit de cada reto y cómo main los incorpora. Luego usa `git log --graph --oneline --all` para comparar tu dibujo con el historial real.